Developing Back-End Apps with Node.js and Express
Module 3 Cheat Sheet: Express Web Application Framework

Package/Method	Description	Code Example
Dependencies in `package.json`	A dependency of **express** version between **4.0 to 5.0** will be declared as:	1. 1 1. "dependencies":{"express":"4.x"} Copied!
new express()	Creates an express object which acts as a server application.	1. 1 2. 2 1. const express = require("express"); 2. const app = new express(); Copied!
express.listen()	The listen method is invoked on the express object with the port number on which the server listens. The function is executed when the server starts listening.	1. 1 2. 2 3. 3 4. 4 1. app.listen(3333, () => { 2. console.log("Listening at 3. http://localhost:3333) 4. }) Copied!
express.get();	This method is meant to serve the retrieve requests to the server. The get() method is to be implemented with two parameters; the first parameter defining the end-point and the second parameter is a function taking the request-handler and response-handler.	1. 1 2. 2 3. 3 4. 4 5. 5 6. 6 1. // handles GET queries to end point /user/about/id. 2. app.get("user/about/:id" , (req,res)=>{ 3. res.send("Response about user " 4. +req.params.id) 5. 6. }) Copied!
express.post();	This method is meant to serve the create requests to the server. The post() method is to be implemented with two parameters: the first parameter defines the end-point and the second parameter is a function taking the request-handler and response-handler.	1. 1 2. 2 3. 3 4. 4 5. 5 1. // handles POST queries to the same end point. 2. app.post("user/about/:id" , (req,res)=>{ 3. res.send("Response about user " 4. +req.params.id) 5. }) Copied!
express.use()	This method takes middleware as a parameter. Middleware acts as a gatekeeper in	1. 1 2. 2 3. 3 4. 4 5. 5 6. 6 7. 7 8. 8

	the same order that it is used, before the request reaches the get() and post() handlers. The order in which the middleware is chained depends on the order in which the .use() method is used to bind them. The middleware myLogger() function takes three parameters, which are request, response, and next. You can define a method that takes these three parameters and then bind it with express.use() or router.use(). Here, you are creating middleware named myLogger and making the application use it. The output rendered includes the time the request is received.	9
		10
		
		`const express = require("express");`
		`const app = new express();`
		`function myLogger(req, res, next){`
		`  req.timeReceived = Date();`
		`  next();`
		`}`
		`app.get("/", (req, res)=>{`
		`  res.send("Request received at`
		`  "+req.timeReceived+" is a success!")`
		`})`
		Copied!

express.Router()	Router-level middleware is not bound to the application. Instead, it is bound to an instance of express.Router(). You can use specific middleware for a specific route instead of having all requests going through the same middleware. Here, the route is /user and you want the request to go through the user router. Define the router, define the middleware function that the router will use and what happens next, and then you bind the application route to the router.	1
		2
		3
		4
		5
		6
		7
		8
		9
		10
		11
		12
		13
		14
		15
		16
		
		`const express = require("express");`
		`const app = new express();`
		`let userRouter = express.Router();`
		`let itemRouter = express.Router();`
		`userRouter.use(function (req, res, next){`
		`  console.log("User quert time:", Date());`
		`  next();`
		`})`
		`userRouter.get("/:id", function (req, res,`
		`  next) {`
		`  res.send("User "+req.params.id+" last`
		`  successful login "+Date())`
		`})`
		`app.listen(3333, () => {`
		`  console.log("Listening at http://localhost:3333)`
		`})`
		Copied!

express.static()	This is an example of static middleware that is used to render	1 2 3 4 5

static HTML pages and the images from the server side. At the application level, the static files can be rendered from the cad220_staticfiles directory. Notice that the URL has only the server address and the port number followed by the filename.

```
6
```

```
const express = require( “express” );
const app = new express();
app.use(express.static("cad220_staticfiles"))
app.listen(3333, () => {
  console.log("Listening at  http://localhost:3333")
})
```

Copied!

jsonwebtoken.sign()

Used for signing-in based on a generated JWT (JSON Web token)

```
1
2
3
4
5
```

```
if (uname === "user" && pwd === "password") {
    return res.json({
      token: jsonwebtoken.sign({ user: "user" }, JWT_SECRET),
    });
  }
```

Copied!

jsonwebtoken.verify()

Verifies a JWT by passing the token value & the JWT secret as arguments.

```
1
```

```
const verificationStatus =   jsonwebtoken.verify(tokenValue, "aVeryVerySecretString");
```

Copied!

Project folder strucure

A fairly established project structure for API's built using Express.js is:

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
```

```
test-project/
   node_modules/
   config/
     db.js           //Database connection and configuration
     credentials.js  //Passwords/API keys for external services used by your app
   models/            //For mongoose schemas
      items.js
      prices.js
   routes/           //All routes for different entities in different files
      items.js
      prices.js
   app.js
   routes.js         //Require all routes in this and then require this file in
   package.json
```

Copied!

Changelog

Date	Version	Changed by	Change Description
04-07-2022	1.0	Pallavi	Initial version created
18-10-2022	1.1	K Sundararajan	Cheatsheet updated
17-11-2022	1.2	K Sundararajan	IDSN logo updated based on Beta feedback
29-11-2022	1.3	K Sundararajan	Title updated based on Beta testing feedback

© IBM Corporation 2022. All rights reserved.